

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Planning the Flutter Web–REST API–PostgreSQL Technology Stack of Your Own Project

Flutter Framework and Technology Stack

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 3
Output	Flutter Web–REST API–PostgreSQL technology stack blueprint, stack-role worksheets, and group presentation
Maximum Score	80 points

Student / Project Information

Block: A-1	Date: 09/18/26
System Title: Seaweed (Guso) Quality Classification Using Image Processing	Instructor: Jean Y. Villasin, MIT
Members: Zydane Angelo Adanza; Jherick Peñalosa; Timothy Josua Tavera	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the prescribed Flutter Web–REST API–PostgreSQL technology stack and justify the role of each component in the proposed web system.

Course Outcome Alignment: CO1 – Explain web architecture and the roles of client-side, server-side, framework, API, database, and deployment components in the end-to-end flow of a web system.

Syllabus Assessment: Flutter–API–PostgreSQL Technology Stack Blueprint; stack-role explanation; online architecture justification task.

Purpose of the Activity

Activity 2 established the overall architecture of your semester project. In Activity 3, your team will convert that architecture into the prescribed implementation stack: Flutter Web for the client interface, a REST API using an instructor-approved backend framework/runtime for application logic, and PostgreSQL as the persistence layer developed in IT 108. You will also identify the role of Dart, Flutter packages, Git/GitHub, and the deployment environment so the blueprint can guide later implementation activities.

Important distinction: Flutter Web is the client application layer; the REST API is the application and integration layer; PostgreSQL is the persistence layer. The Flutter client must not contain database credentials or issue raw SQL directly to PostgreSQL in the prescribed stack.

2. Intended Learning Objectives

- Distinguish a framework, library/package, runtime, API style/protocol, DBMS, version-control tool, and deployment platform.
- Explain the responsibilities of Flutter Web, the REST API/backend, and PostgreSQL in the team's proposed system.
- Identify the Dart/Flutter fundamentals, widgets, packages, state-management approach, and client-side validation needed by major project features.
- Select and justify an instructor-approved backend framework/runtime for the REST API without shifting database or client responsibilities into the wrong layer.
- Map REST API resources and operations to the PostgreSQL tables, relationships, constraints, and transactions being developed in IT 108.
- Create a system-specific Flutter Web–REST API–PostgreSQL technology stack blueprint that includes Git/GitHub, security boundaries, and deployment targets.
- Explain and defend why each technology belongs in the stack and how the components communicate.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the technology decisions and be able to explain one assigned stack component.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Flutter Web–REST API–PostgreSQL Technology Stack Blueprint.

Activity Item	Requirement
Supporting Output	Technology classification worksheet, stack-role matrix, Flutter client plan, REST API plan, PostgreSQL mapping, Git/GitHub and deployment plan, 120–180 word stack-role explanation, and group presentation.

Recommended Tools	diagrams.net/Draw.io, Lucidchart, Figma, Canva, PowerPoint, Git/GitHub, Flutter/Dart documentation, backend API documentation, and PostgreSQL documentation.
Presentation Format	5–7 minutes per group followed by 2–3 minutes of questions. Every member must explain an assigned technology, interface, or integration decision.

4. Pre-Laboratory Requirements

- Final or revised architecture diagram from Laboratory Activity No. 2.
- Approved system title, primary users/roles, and at least five major system features.
- Current ERD or normalized PostgreSQL schema from IT 108, including primary keys, foreign keys, and important constraints.
- Instructor-approved backend framework/runtime for the REST API. If approval is still pending, bring no more than two candidate options for consultation.
- Initial list of Flutter packages/state-management tools and external services that the project genuinely needs.
- Access to the project GitHub repository and the team's planned deployment environment or hosting options.
- Copies of Flutter/Dart, REST API/backend, and PostgreSQL documentation relevant to the selected technologies.

5. Part A – Technology Classification and Role Check

Before assigning technologies to the project, classify each item correctly. Use the following distinctions: **framework** provides an application structure and conventions; **library/package** provides reusable functions or components; **runtime** executes application code; and the remaining categories describe communication, persistence, version control, or hosting responsibilities.

Category	Meaning
Framework	Provides structure, conventions, lifecycle, and reusable building blocks for an application.
Library / package	Reusable code added to the application for a focused capability such as HTTP, state management, charts, maps, or validation.
Runtime	Environment that executes application or server code.
API style / protocol / data format	Defines how components communicate and how requests/responses are represented.
DBMS	Manages persistent structured data, constraints, transactions, and queries.
Version control / collaboration	Tracks changes, supports branches and reviews, and stores project history.

Deployment platform / environment Runs or hosts the client, API, database, and supporting services.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3 Classification Worksheet

Technology / Concept	Classification	Role in Your Project	Project-Specific Choice / Evidence
Dart	Programming Language	Used to write the Flutter Web application and client-side application logic.	Dart is the programming language used by Flutter.
Flutter	Framework	Provides the structure, widgets, UI components, navigation, and responsive interface for the web client.	Flutter Web is used for the web-based Guso classification system.

Flutter package for HTTP/API calls	Library / Package	Sends HTTP/HTTPS requests from Flutter Web to the FastAPI REST API.	Team-selected HTTP/API package; confirm the exact package before submission.
Riverpod	State-management package / pattern	Manages application state such as login state, uploaded image, loading state, classification result, history, and errors.	Riverpod is the selected Flutter state-management approach.
REST + HTTPS + JSON	API style / protocol / data format	Defines how Flutter Web and FastAPI communicate through requests and responses.	REST API over HTTPS; JSON for API data. Image upload may use multipart/form-data.
Python	Programming Language	Used to implement the FastAPI backend.	Python is the backend language.
FastAPI	Framework	Handles API routes, authentication, validation, business/classification logic, and communication with PostgreSQL.	FastAPI REST API is the selected backend framework.
Python runtime	Runtime	Executes the FastAPI/Python backend application.	Python runtime/environment runs the FastAPI server.
PostgreSQL	DBMS	Stores users, Guso images, classification results, and history/records.	PostgreSQL is the persistence layer.
Git / GitHub	Version control / collaboration	Stores source code, tracks changes, supports branches, and enables team collaboration.	Group GitHub repository for Flutter, FastAPI, database scripts, and documentation.

6. Part B – Prescribed Stack Role Matrix

Complete the matrix using the exact technologies planned for your system. The “Must Not Be Responsible For” column is included to prevent duplicated logic and insecure shortcuts.

Layer / Area	Prescribed Technology Your Project-Specific Choice	Main Responsibility	Must Not Be Responsible For
Presentation / Client	Flutter Web + Dart Flutter Web + Dart	Screens, widgets, navigation, state, form handling, user feedback	Database credentials; raw SQL; final authorization rules

Client packages	Team-selected Flutter packages	Riverpod + team-selected HTTP/API package	HTTP/API calls, state management, validation, maps/charts, or other approved client functions	Server-only secrets or database administration
Integration	REST API over HTTPS using JSON	REST API over HTTPS using JSON	Defines endpoints, request/response formats, status codes, and client-server communication	Persistent data storage by itself
Application / Backend	FAST API / Python FastAPI / Python		Authentication/authorization checks, validation, business rules, orchestration, API response generation	Rendering the Flutter UI; storing production data in local code files
Persistence	PostgreSQL schema developed in IT 108	PostgreSQL	Tables, relationships, constraints, transactions, queries, durable records	User-interface logic or route navigation
Version control	Git + GitHub		Source history, collaboration, branches, pull requests, issue/document tracking	Running the production system
Deployment	Team-selected hosting / server / managed DB	Selected hosting platforms	Runs Flutter Web, API, PostgreSQL, and environment configuration	Replacing application-layer security or database

			constraints
--	--	--	-------------

Stack-role rule

A good blueprint separates concerns. Flutter Web handles the user experience and client state; the REST API/backend protects and processes requests; PostgreSQL persists validated data and enforces relational integrity. Git/GitHub supports collaboration, while deployment platforms host the running components. If a responsibility appears in two layers, the team must explain why it is not duplicated or contradictory.

7. Part C – Flutter Web Client Blueprint

Plan the client side around your actual system features. List at least five important screens or feature modules. Use package names only when the team has decided to use them.

Feature / Screen	Main Widgets / Forms	State / Controller	Flutter Package(s)	API Operation Needed	Client Validation / User Feedback
Login	TextFields, password field, Login button	Riverpod Auth State	HTTP package + Riverpod	POST /api/auth/login	Required fields; show loading and login errors.
Register	Registration form, TextFields, Register button	Riverpod Auth State	HTTP package + Riverpod	POST /api/auth/register	Required fields and password validation.

Dashboard	Cards, navigation buttons, summary widgets	Riverpod DashboardState	Riverpod	GET /api/users/me or dashboard endpoint	Loading, error, and empty states.
Guso Quality Classification	Image upload, preview, Classify button	Riverpod Classification State	HTTP package + Riverpod	POST /api/classifications	Validate image type/size; show upload and loading feedback.
Login	TextFields, password field, Login button	Riverpod Auth State	HTTP package + Riverpod	POST /api/auth/login	Required fields; show loading and login errors.
Register	Registration form, TextFields, Register button	Riverpod Auth State	HTTP package + Riverpod	POST /api/auth/register	Required fields and password validation.

Client planning checklist:

- Routing/navigation between screens is identified.
- Responsive web layout and loading/error/empty states are considered.
- Client-side validation improves usability but does not replace server-side validation.
- No database password, private API key, or other server secret is embedded in Flutter Web source code.

8. Part D – REST API and Backend Plan

Selected backend framework: FastAPI

Selected backend runtime: Python

Why this choice fits the project (3–5 sentences):

FastAPI is suitable for our system because it allows us to build REST API endpoints that connect the Flutter Web client to the backend. It can handle authentication, authorization, request validation, image processing, and business logic in one application layer. Our MobileNetV3 classification model is integrated into the FastAPI backend, allowing uploaded Guso images to be processed and classified without requiring a separate external ML service. FastAPI can also communicate with PostgreSQL to store and retrieve classification records.

Minimum API Contract Worksheet

Define at least six REST operations that support the project's real workflows. Use an instructor-approved endpoint naming convention and appropriate HTTP methods.

Use Case	Method	Endpoint	Auth / Role	Request Data	Server Action / Rule	Response / Status
Register account	POST	/api/auth/register	Public	User registration data	Validate data and create account.	JSON + 201 Created
Login	POST	/api/auth/login	Public	Username/email + password	Verify credentials and issue JWT.	JSON + 200 OK
Get current user	GET	/api/users/me	JWT	Token	Verify JWT and return user information.	JSON + 200 OK

Classify Guso image	POST	/api/classifications	JWT + role	Guso image	Validate image, preprocess, run MobileNetV3, and save result.	JSON + 201/200
View history	GET	/api/classifications/history	JWT	Optional filters	Retrieve user's classification records.	JSON + 200 OK
View classification record	GET	/api/classifications/{id}	JWT + ownership/role	Record ID	Verify access and retrieve record.	JSON + 200 OK
Download/save record	GET	/api/classifications/{id}/download	JWT + ownership/role	Record ID	Generate/return downloadable record.	File/JSON + 200 OK

REST API guidance

The API should validate input, verify identity and role permissions, apply business rules, interact with PostgreSQL or approved external services, and return an appropriate HTTP status with JSON data. Common methods include GET, POST, PUT/PATCH, and DELETE. The exact endpoints and rules must come from your own project requirements.

9. Part E – PostgreSQL Persistence and IT 108 Integration

Map the API resources to the PostgreSQL schema being designed in IT 108. Do not invent a second unrelated database model for IT 110. Record important keys, constraints, and transaction rules that affect the API.

API Resource / Feature	PostgreSQL Table(s)	PK / FK / Constraint	DML Operation	Transaction / Integrity Rule	Returned Fields / Result
User Registration	Users	User ID as PK; unique login identifier	INSERT	Prevent duplicate accounts.	User ID / account information
Login / User Profile	Users	User ID PK	SELECT	Return appropriate account data.	User information
Guso Image Upload	Guso Images	Image ID PK; User ID FK	INSERT	Image must belong to valid user.	Image ID / upload information
Classification Result	Classification Results	Result ID PK; Image/User FK	INSERT	Result must be associated with valid image/user.	Classification result
History / Records	History / Records	Record ID PK; appropriate FKs	SELECT	User accesses only permitted records.	Classification history
Record Details	Classification/History tables	PK/FK relationships	SELECT	Verify record ownership/access.	Selected record
Delete/Update Record (if supported)	Relevant table	PK/FK constraints	UPDATE/DELETE	Preserve referential integrity.	Success/status

Database boundary rule

In the prescribed Flutter Web–REST API–PostgreSQL stack, Flutter Web should not connect directly to PostgreSQL. The backend owns database credentials and uses the database driver/ORM or approved data access layer. PostgreSQL remains responsible for persistent records, keys, constraints, transactions, and query execution.

10. Part F – Git/GitHub and Deployment Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Repository ownership	Group repository URL, members with access, and repository visibility as instructed	Group GitHub repository for the Guso Quality Classification project.
Folder structure	Identify where Flutter client, backend API, documentation, and database scripts/migrations will be stored	frontend/ for Flutter Web; backend/ for FastAPI; database/ for PostgreSQL scripts/migrations; docs/ for architecture and documentation.
Branching / collaboration	Main branch, feature branches, commit/merge practice, and review responsibility	Main branch plus feature branches; members commit and merge changes after review.
Environment variables	List values that must be configured outside source code, such as API base URL and server/database secrets	API base URL, database connection string, JWT secret, and other server secrets stored outside source code.
Flutter Web deployment REST API deployment PostgreSQL deployment	Hosting platform or server for the built web client Server, container, cloud runtime, or approved hosting platform Local development instance and planned managed/server production database	Vercel or Cloudflare Pages — use the final provider selected by the group. Render or Railway — use the final provider selected by the group. Managed PostgreSQL/database hosting; local PostgreSQL for development.
Logging / backup / recovery	Minimum plan for logs, error diagnosis, database backup, and restoration	Backend logs for errors; regular PostgreSQL backups; restoration procedure for production data.

Recommended minimum repository organization (adapt as needed):

```
project-root/  
  frontend/ Flutter Web application  
  backend/ REST API source code  
  database/ PostgreSQL scripts or migrations, when used  
  docs/ architecture and technology-stack blueprints  
  README.md  
  .gitignore  
  .env.example variable names only; never commit real secrets
```

11. Part G – Construct the Technology Stack Blueprint

Create one clean, readable blueprint that shows how the prescribed stack is implemented in your own system. The blueprint must be understandable without opening the source code.

No.	Required Element	What Must Be Shown
1	System identity	Official project title and “Flutter Web–REST API–PostgreSQL” stack label.
2	Users / actors	Major users who interact with the Flutter Web client.
3	Flutter Web client	Dart, key screens/widgets, routing, state management, form validation, and HTTP/API client.

No.	Required Element	What Must Be Shown
4	Flutter packages	Only packages actually planned for the project; state their purpose.
5	Communication	HTTPS, REST endpoints, JSON request/response, and API base URL/environment configuration.
6	Backend framework/runtime	Exact instructor-approved framework and runtime used to implement the API.
7	Application logic	Authentication, authorization, validation, services, business rules, and external-service integration.
8	PostgreSQL persistence	Relevant tables/data domains from IT 108, keys/constraints, and database access mechanism.
9	Git/GitHub	Repository and collaboration boundary for client, backend, database scripts, and documentation. Where the Flutter build, REST API, PostgreSQL database, and storage/external services will run. Show where secrets live, where tokens/roles are checked, and which resources are protected.
10	Deployment	
11	Security boundary	
12	Data flow	Directional arrows labeled with requests, responses, JSON, SQL/data access, and returned results.

Minimum Blueprint Rule

Your blueprint must show at least one complete implementation path: User → Flutter Web interface → HTTPS/JSON REST request → backend API logic → PostgreSQL transaction/query → JSON response → Flutter state/UI update. Include Git/GitHub and deployment information as supporting infrastructure, not as steps in the runtime request path.

12. Reference Example – Do Not Copy as Your Final Output

The figure below shows the level of stack separation and labeling expected. Replace every generic label with the technologies, features, endpoints, and database objects used by your own project.

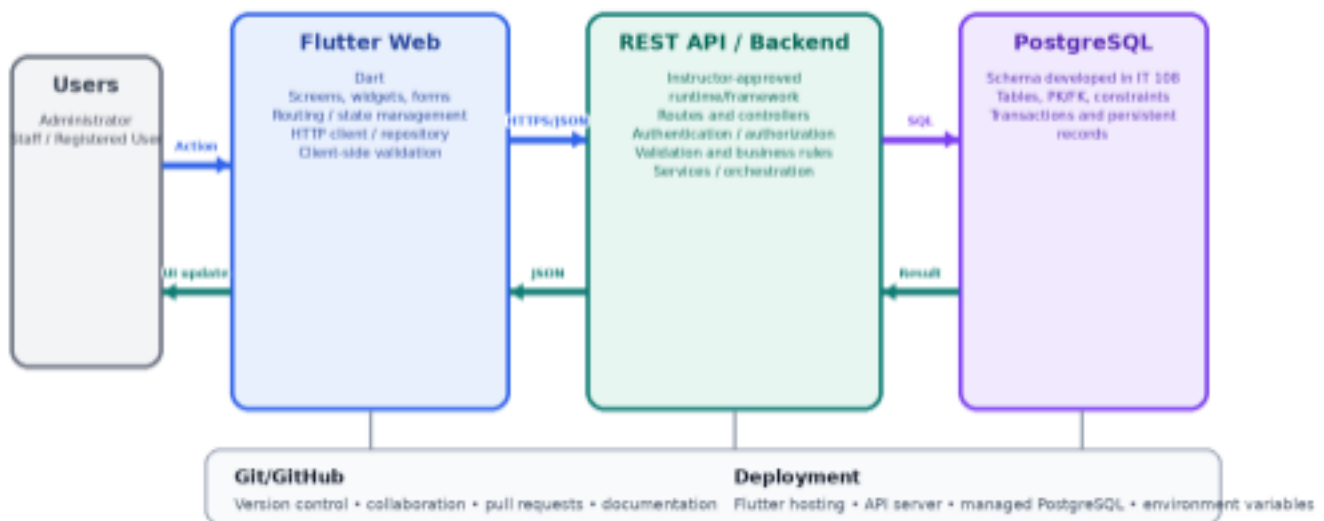
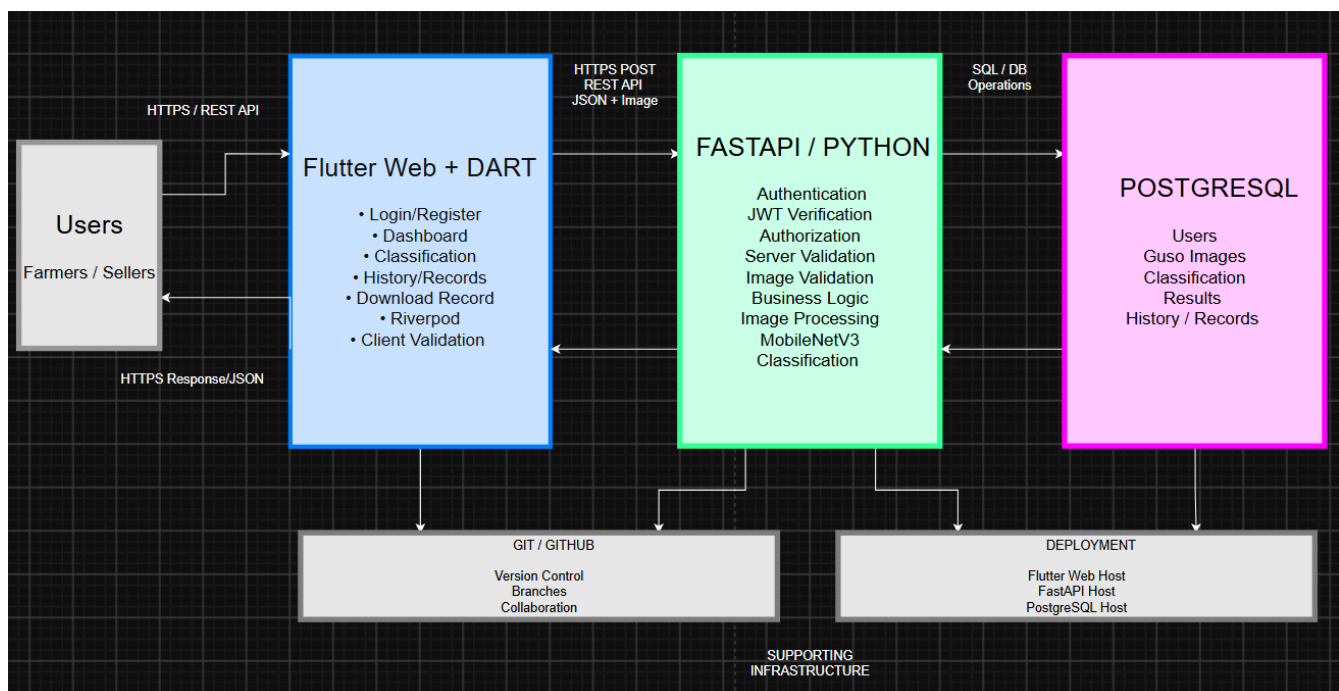


Figure 1. Reference Flutter Web–REST API–PostgreSQL technology stack and runtime path.

Step	Reference Flow
1	User interacts with a Flutter Web screen.
2	Flutter validates the form and sends an HTTPS request containing JSON.
3	The REST API verifies access, validates data, and executes business rules.
4	The backend queries or updates PostgreSQL using an approved data-access mechanism.
5	The API returns an HTTP status and JSON payload.
6	Flutter updates state and displays the result to the user.



13. Part H – Feature-to-Stack Responsibility Trace

Selected workflow / feature: Guso Quality Classification

Expected result shown to the user: The system displays the predicted Guso quality classification and related classification information.

Step	Responsible Component	Action / Data	Why It Belongs in This Component	Evidence in Blueprint
1	Farmer/Seller	Selects and uploads Guso image.	User initiates the transaction.	Actors
2	Flutter Web	Displays upload interface and performs basic client validation.	User-interface responsibility.	Flutter Web
3	Riverpod	Manages selected image, loading state, result, and errors.	Client state-management responsibility.	Riverpod
4	Flutter HTTP/API	Sends image and request to FastAPI through HTTPS.	Client-to-server communication.	REST API
5	client FastAPI	Receives API request.	Backend handles API requests.	FastAPI
6	FastAPI Security	Verifies JWT and user authorization.	Security is enforced server-side.	JWT/Auth
7	FastAPI Validation	Validates uploaded image/request.	Client validation cannot be trusted alone.	Server-side validation
8	Business/Classification Logic	Processes the image.	Backend owns business rules and processing.	Classification Logic
9	MobileNetV3	Classifies the processed Guso image.	ML model performs classification.	MobileNetV3

10	PostgreSQL	Stores image/result/history.	Database owns persistent records.	PostgreSQL
11	FastAPI	Retrieves/stores classification information and creates response.	Backend connects application to database.	FastAPI
12	Flutter Web + Riverpod	Receives JSON and updates UI.	Client displays returned information.	Flutter/Riverpod
13	Farmer/Seller	Views classification	Final user-facing	Flutter UI

		result.	result.	
--	--	---------	---------	--

14. Part I – Stack-Role Explanation

Write 120–180 words explaining and justifying the prescribed technology stack for your own system. Address all of the following:

- What Flutter Web and Dart are responsible for in the user experience.
- Why the REST API/backend is the correct place for authentication/authorization checks, server-side validation, and business rules.
- How PostgreSQL persists the data model being developed in IT 108 and protects relational integrity.
- Why Git/GitHub and the deployment environment are necessary even though they are not runtime application layers.
- How the separation improves security, maintainability, testing, integration, and future scaling, including one implementation risk the team must manage.

Our system uses the prescribed Flutter Web–REST API–PostgreSQL technology stack to implement the Seaweed (Guso) Quality Classification Using Image Processing system. Flutter Web and Dart are responsible for the user interface, including the dashboard, image upload, classification result, and history features. Riverpod manages client-side state and helps update the interface when data changes. The FastAPI REST API using Python serves as the application layer, where authentication, JWT verification, authorization, server-side validation, business rules, image processing, and MobileNetV3 classification are handled. PostgreSQL serves as the persistence layer for user accounts, Guso images, classification results, and history records while maintaining relational integrity. Git and GitHub support source-code management and team collaboration, while deployment platforms host the running application and services. Separating these responsibilities improves security, maintainability, testing, and future scalability because the client does not directly access the database. One implementation risk is image-processing latency between FastAPI and MobileNetV3, which could cause slow responses or request timeouts.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The official system title and actual project features are used.	☐ The stack is explicitly labeled Flutter Web–REST API–PostgreSQL.
☐ Framework, library/package, runtime, DBMS, version control, and deployment platform are classified correctly.	☐ Flutter screens/widgets, client state, packages, and API calls are project-specific.
☐ The instructor-approved backend framework/runtime is named.	☐ At least six meaningful REST operations are defined.
☐ Authentication, authorization, validation, and business rules are located primarily in the backend/API layer.	☐ The PostgreSQL mapping matches the IT 108 schema or clearly identifies coordinated schema changes.
☐ Flutter Web does not contain database credentials or direct raw SQL access to PostgreSQL.	☐ HTTPS/REST/JSON and database-access arrows are directional and meaningfully labeled.
☐ Git/GitHub organization, environment variables, and deployment targets are identified.	☐ At least one complete User → Flutter → API → PostgreSQL → API → Flutter path is visible.

☐ The written stack-role explanation agrees with the blueprint and worksheets.	☐ Every group member can explain one component and defend the complete integration path without reading a script.
---	--

16. Submission Requirements

- PDF or PNG of the final technology stack blueprint in readable landscape orientation.
- Editable blueprint source file such as .drawio, .pptx, .fig, or the native file of the selected diagramming tool.
- Completed Parts A–H and the 120–180 word stack-role explanation.
- Updated project GitHub repository containing the blueprint and supporting files under a folder such as docs/technology-stack/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA3_SystemName.
- Deliver a 5–7 minute group presentation using the final blueprint as the main visual, followed by 2–3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own system and technology decisions. AI tools may be used to clarify terminology or review ideas, but the final stack classification, framework/runtime selection, endpoint plan, PostgreSQL mapping, blueprint, and justification must be verified, understood, and defended by the team. The instructor may ask any member to explain why a component belongs in a particular layer or trace a request without assistance.

17. Scoring Rubric – 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unequal participation, excessive reading, incorrect role assignment, weak integration reasoning, or inability to answer questions about the team's own stack.

Criterion	Evidence for Full Credit	Points
-----------	--------------------------	--------

F-CID-010 Rev.3, 03-21-2025 Competent • Service-oriented • Upright Page 10 of 12
CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 3

Criterion	Evidence for Full Credit	Points
Framework/library/runtime and stack-role accuracy	Technologies are classified correctly, and the team clearly distinguishes Flutter, Dart, packages, REST, backend runtime/framework, PostgreSQL, Git/GitHub, and deployment responsibilities.	10
Flutter Web client planning	Screens/features, widgets/forms, state approach, packages, client validation, API calls, loading/error feedback, and client security boundaries are complete and project-specific.	10
REST API and backend plan	An instructor-approved backend framework/runtime is justified; endpoints, methods, authentication/roles, validation, business rules, request data, responses, and status behavior are coherent.	14
PostgreSQL persistence integration	API resources map correctly to the IT 108 schema; important tables, PK/FK constraints, DML operations, transactions, and integrity requirements are represented accurately.	11

Technology stack blueprint and end-to-end integration

The blueprint clearly integrates users, Flutter Web, packages/state,

HTTPS/REST/JSON, backend application logic, PostgreSQL, security, and a complete request-response path.

Git/GitHub, deployment, and secret management	Repository organization, collaboration approach, environment variables, hosting targets, and protection of server/database secrets are identified and realistic.	5
Written stack-role explanation	The 120–180 word explanation accurately justifies component roles, separation of concerns, IT 108 integration, security, maintainability, testing, deployment, scaling, and one implementation risk.	5
Group presentation and oral defense	Presentation is clear, organized, within time, and uses the final blueprint effectively; all members participate and correctly defend technology roles, API/database boundaries, and the selected workflow.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which responsibility was hardest for your group to assign correctly among Flutter Web, the REST API, and PostgreSQL, and why?

The hardest responsibility to assign was the classification and validation process because we needed to distinguish between client-side validation and backend business logic. Flutter Web can perform basic image validation for better user experience, but the FastAPI backend must perform the actual server-side validation and classification. PostgreSQL is responsible for storing and retrieving persistent data.

Which Flutter package or client-side pattern is most important to your project, and what problem will it solve?

Riverpod is the most important client-side pattern for our project because it manages the application's state. It helps manage the uploaded image, loading status, classification result, user information, history records, and errors. This allows the Flutter Web interface to update properly when data changes.

What server-side rule must never rely only on Flutter client validation?

Authentication, authorization, and image validation must never rely only on Flutter client validation. The FastAPI backend must verify the user's JWT, check the user's permissions, validate the uploaded image, and apply the system's business rules because client-side validation can be bypassed.

Which PostgreSQL constraint or transaction rule from IT 108 is most important to preserving correct system data?

Primary key and foreign key relationships are important because they ensure that classification results and Guso images are associated with valid users and related records. The exact constraint names should follow our IT 108 ERD.

What must your team configure before the first real Flutter-to-REST-API integration test can succeed?

We need to configure the FastAPI server, API base URL, PostgreSQL database connection, required environment variables, authentication/JWT configuration, and the Flutter HTTP/API client. The Flutter Web client must be able to reach the FastAPI endpoint through the configured development or HTTPS connection.

What is the first stack component your group should implement after this activity, and what evidence will prove that it works?

The first component should be the FastAPI REST API foundation because it connects the Flutter Web client with PostgreSQL and the classification logic. Evidence that it works would be a successful API request from Flutter Web to FastAPI, proper authentication/validation, a successful database operation, and a JSON response displayed in the Flutter interface.